Table of Contents

Introduction

Background and Project Concept

Financial markets have evolved into fast-moving, data-rich environments where informed decision-making requires both speed and technical expertise. Retail investors, in particular, face significant challenges when navigating market volatility, interpreting technical indicators, and understanding the logic behind algorithmic trading systems. At the same time, the increasing availability of financial data and the rapid advancement of machine learning have created opportunities to support these users with intelligent tools that simplify complex analytical processes. However, most algorithmic trading systems remain inaccessible to non-technical audiences and often operate as "black boxes," providing predictions without clear explanations.

The Financial Advisor Bot project is being developed to address this gap. It aims to create an explainable, AI-assisted stock-trading decision-support system that can generate BUY, SELL, or HOLD signals by combining classical technical analysis with reinforcement learning (RL). Rather than functioning as a fully autonomous trading algorithm, the bot is intended to support human decision-making by offering interpretable insights into market conditions. The core concept is to process historical stock price data, detect short-term and medium-term market patterns, and translate these into actionable recommendations accompanied by human-readable explanations.

At this stage of development, the project includes the early implementation of a Simple Moving Average (SMA) crossover signal generator, which forms the baseline strategy for the system. This prototype is being used to test feasibility, data processing pipelines, and signal accuracy. The more advanced RL-based system, natural-language explanation engine, backtesting module, and the full interactive web interface are in progress and will be developed iteratively over subsequent project phases.

Overall, the project is positioned at the intersection of financial technology, artificial intelligence, and explainable systems. It aims not only to generate trading signals but also to make the underlying reasoning transparent and accessible to users with limited financial or technical experience.

Motivation

The motivation for this project arises from a combination of market demand, technical challenges, and educational value. Financial decision-making traditionally favours institutional investors who possess access to sophisticated modelling tools, large datasets, and automated trading systems. Retail investors, in contrast, often rely on instinct, fragmented online resources, or visually complex charting tools. This imbalance has widened as hedge funds increasingly deploy algorithmic strategies capable of detecting micro-patterns at speeds unattainable for humans.

Additionally, while numerous platforms provide market data or chart visualisations, few tools offer explainable trading insights tailored to non-experts. Many existing financial bots and trading tools provide raw predictions or black-box model outputs without clarifying the logic behind them. This lack of interpretability can lead to distrust, over-reliance, or misinformed decision-making.

From a technical standpoint, the project presents an opportunity to investigate how reinforcement learning can be applied in financial environments characterised by uncertainty, sequential decision processes, and dynamic state changes. Reinforcement learning aligns well with trading contexts but requires careful design and evaluation to avoid overfitting and ensure interpretability.

On a personal and academic level, the project offers a practical application of machine-learning techniques, software engineering principles, and user-experience design. It allows exploration of how AI-driven solutions can be made transparent and accessible to broader communities, especially users without backgrounds in algorithmic trading or data science.

Project Aims and Objectives

The overarching aim of the Financial Advisor Bot is to build an interactive, explainable advisory system that supports stock-trading decisions through a combination of reinforcement learning and classical technical indicators. To achieve this aim, the project is guided by the following objectives:

Primary Objectives

1. Develop a hybrid decision-making engine that integrates a Q-learning reinforcement learning agent with traditional technical indicators such as SMA crossovers.
2. Implement a natural-language explanation component capable of converting model outputs into understandable interpretations tailored for non-expert users.
3. Design a responsive web-based interface that enables users to query stock symbols and receive real-time recommendations in a conversational format.
4. Develop a backtesting module to simulate strategy performance using historical data and provide quantitative evaluation metrics.
5. Ensure accessibility and interpretability so that users can understand the reasoning behind each BUY, SELL, or HOLD signal.

Current Progress

- The SMA crossover strategy has been partially implemented and is capable of generating preliminary BUY/SELL/HOLD signals.
- A simple data retrieval pipeline using yfinance is functional.
- Early drafts of the system architecture and module interactions have been designed.
- The RL agent, explanation engine, full backtesting module, and expanded frontend are in active planning or early development.

Long-term Objectives

- Integrate additional technical indicators and confidence-scoring mechanisms.
- Evaluate the RL agent against the SMA baseline through comparative backtesting.
- Enhance the explanation engine using hybrid template-based and AI-generated approaches.
- Conduct user-focused testing to assess interpretability and usability.

Template Selection: Decision-Making Under Uncertainty

This project adopts the Decision-Making Under Uncertainty template, a framework well-aligned with environments where outcomes are affected by incomplete information, market volatility, and sequential decision processes. Stock markets are inherently uncertain: investors cannot fully predict future prices, and decisions must be made using partial or noisy information.

Reinforcement learning is particularly suited to this template. It models decision-making as interactions between an agent and an environment, where the agent learns an optimal policy through exploration, rewards, and iterative improvement. This mirrors real-world trading behaviour, where a sequence of actions - buying, holding, or selling - affects future opportunities and risks.

The SMA strategy prototype already demonstrates this principle by analysing patterns of moving averages under uncertain conditions. As the project progresses, the RL agent will further embody the Decision-Making Under Uncertainty template, learning to adapt to complex market states through experiential feedback.

Scope of This Preliminary Report

This preliminary report reflects the project's status at the mid-development stage. It includes:

- A revised introduction with updated aims and motivations
- A developing literature review informed by ongoing research
- A design chapter outlining the architectural plan and work breakdown
- A feature prototype chapter focusing on the early implementation of the SMA crossover strategy

The report does not yet present final evaluation results, a complete RL model, or the fully integrated system. Instead, it establishes the foundation for further development, testing, and refinement in the next stages of the project.

Literature Review

Introduction

The development of algorithmic trading and intelligent financial advisory systems has grown significantly over the past two decades due to advances in data availability, computational power, and machine-learning techniques. These systems increasingly shape investor behaviour and market dynamics, yet their complexity frequently creates barriers for non-expert users. Retail investors often lack the domain knowledge needed to interpret market signals or understand how algorithmic strategies operate, highlighting a persistent gap between sophisticated trading algorithms and accessible decision-support tools.

This literature review critically examines academic and industry research across five key areas that inform the design and direction of the Financial Advisor Bot:

1. Reinforcement learning (RL) in financial trading
2. Moving average–based technical analysis
3. Backtesting and strategy validation
4. Explainable AI (XAI) and natural language generation (NLG)
5. Existing FinTech tools and advisory systems

By analysing both foundational and contemporary studies, this chapter identifies the limitations of current approaches and positions the Financial Advisor Bot within a broader research context. The insights extracted from this literature are actively informing the system's architecture and development, particularly as the project is still in mid-implementation.

Reinforcement Learning in Financial Trading

Reinforcement learning has been widely explored in financial modelling due to its ability to optimise sequential decisions under uncertainty - an inherent characteristic of stock markets. The agent–environment framework makes RL suitable for dynamic contexts

where returns depend on multiple time-dependent actions rather than isolated predictions.

Foundational Work

Li, Yang, and Zhang (2009) introduced one of the earliest applications of Q-learning for equity trading. Their tabular RL agent learned BUY/SELL actions by maximising cumulative reward using historical price data. While their results showed promise - particularly during volatile periods - the approach lacked mechanisms for scalability and interpretability. Since the model was limited to discrete state spaces and did not integrate technical indicators, its practical applicability for retail users remained constrained.

Moody and Saffell (2001) expanded on this direction by proposing a recurrent reinforcement learning (RRL) framework aimed at optimising financial returns using differential Sharpe ratio optimisation. Their work demonstrated that RL could incorporate risk-adjusted performance measures, offering more robust portfolio management. However, the mathematical complexity of RRL made it difficult for non-experts to understand, and the model offered little transparency regarding decision pathways.

Deep RL Approaches

The emergence of deep RL models further advanced this area. Jiang, Xu, and Liang (2017) applied a Deep Q-Network (DQN) to cryptocurrency markets, enabling the agent to extract patterns directly from raw price histories without manual feature engineering. Their model outperformed several baselines but required significant computational resources and suffered from issues such as unstable training and sensitivity to hyperparameters - challenges commonly observed in deep RL. Moreover, deep RL systems typically behave as black boxes, offering minimal interpretability for users seeking actionable insights.

Current Relevance to the Project

The Financial Advisor Bot draws heavily on insights from these studies. Their findings highlight both the opportunities and pitfalls of RL-based financial systems:

- **Strength:** Ability to adapt to changing market conditions
- **Limitation:** Lack of interpretability, potential for overfitting, and high computational demand

Given the project's focus on explainability and accessibility, a tabular Q-learning approach is currently being explored as a more transparent and interpretable alternative. This aligns with the project's early-stage goals to prioritise user trust and clarity over predictive complexity. The RL component is still in development, but the reviewed literature forms the conceptual foundation for its planned integration.

Moving Average - Based Strategies in Technical Trading

Technical analysis remains one of the most widely used approaches for market interpretation, particularly due to its simplicity, interpretability, and empirical effectiveness. Among these methods, Simple Moving Average (SMA) strategies are foundational tools for identifying price trends and momentum shifts.

Empirical Foundations

Brock, Lakonishok, and LeBaron (1992) conducted a landmark study demonstrating that even simple moving-average crossover strategies - such as the widely used 50-day vs 200-day SMA - can generate returns exceeding those of random-walk models. Their work provided evidence that markets exhibit patterns that can be exploited through systematic technical rules. However, their approach lacked adaptability and could not dynamically respond to regime shifts, making it vulnerable during unusually volatile markets.

Neely et al. (2009) further validated the profitability of moving-average strategies in foreign exchange markets, showing that such signals can identify momentum in asset prices. Yet, they also acknowledged diminishing returns when transaction costs or rapid

market changes were factored in, suggesting that static rule-based strategies may underperform without adaptive components.

Hybrid Approaches

More recent research incorporates technical indicators into machine-learning models. Patel et al. (2015), for example, used moving averages as engineered features within ensemble learners for stock movement prediction. Although these methods improved accuracy, they introduced black-box elements that compromised interpretability - an issue that the Financial Advisor Bot aims to avoid.

Current Relevance to the Project

Given its simplicity and transparency, the SMA crossover technique is being used as the first implemented prototype in this project. It provides:

- Interpretability suitable for non-experts
- A benchmark for evaluating later RL-based strategies
- A clear foundation for early testing and user feedback

This approach aligns with the project's explainability goals and serves as a baseline for more advanced components.

Backtesting Methodologies and Strategy Validation

Backtesting is essential for evaluating the feasibility and performance of algorithmic trading strategies. It enables developers to test models against historical data, identify weaknesses, and assess risk exposure before deployment in real markets.

Academic Frameworks

Lo, Mamaysky, and Wang (2000) proposed one of the earliest pattern-recognition–driven backtesting frameworks, mapping resulting trading signals to historical returns. Their method demonstrated how structured validation can link technical patterns with financial performance. However, their approach assumed stable relationships between indicators and returns - an assumption that may not hold in highly dynamic markets.

Modern Backtesting Platforms

Tools like Backtrader (2023) and QuantConnect (2024) offer extensive simulation capabilities including order execution modelling, slippage, and multi-asset backtesting. While powerful, these platforms involve steep learning curves and often require complex configuration. Their design suits professionals rather than beginners, limiting usability for educational or casual retail contexts.

Current Relevance to the Project

The Financial Advisor Bot is developing a lightweight, custom backtesting module to:

- Validate early SMA signals
- Support iterative RL training
- Present results in user-friendly formats

This module is currently in partial development, with preliminary simulations confirming that the SMA logic produces plausible trend-based signals. Insights from the literature highlight the importance of avoiding look-ahead bias, accounting for transaction costs, and ensuring reproducibility - all of which are guiding the ongoing refinement of the project's backtesting component.

Explainable AI (XAI) and Natural Language Generation in Finance

As AI models become increasingly complex, explainability has emerged as a critical requirement for user trust - especially in financial contexts where decisions directly impact monetary outcomes.

Early XAI Approaches

Krause, Perer, and Ng (2016) demonstrated the value of template-based explanations in fraud detection systems. Their work showed that converting model decisions into clear natural-language statements significantly improved user comprehension and reliance. However, the explanations were static and non-adaptive, limiting applicability in contexts requiring dynamic interpretation, such as trading.

LLM-Based Explanation Models

Chen, Zhang, and Wang (2021) explored the use of GPT-style large language models to summarise financial insights from quantitative signals. While linguistically fluent, these models were prone to hallucination and lacked domain constraints - raising concerns about reliability and factual accuracy, especially when dealing with risk-sensitive decisions.

Hybrid NLG Models

Hybrid approaches such as the model proposed by Haque and Chen (2022) combine template-based logic with contextual variables to produce explanations that are both stable and adaptable. This has implications for systems requiring partial automation with minimal unpredictability.

Current Relevance to the Project

The Financial Advisor Bot plans to incorporate a two-layer explanation engine:

- **Primary Layer:** Template-based NLG for deterministic explanations
- **Optional Layer:** LLM-powered variation for more natural phrasing

The literature clearly indicates that explainability is essential for user trust and that templated explanations offer the most reliable starting point. Therefore, the explanation module is under design but not yet implemented in full; insights from XAI literature are currently shaping its structure and constraints.

Existing Stock Advisor Bots and FinTech Tools

Examining existing FinTech systems helps position the Financial Advisor Bot within a competitive and technological context.

Retail Investment Platforms

Robinhood and eToro have broadened market accessibility by providing simplified trading interfaces and social investment features. However, their recommendations often rely on crowd behaviour or unstructured sentiment, offering little insight into decision logic. Their lack of transparent explanation mechanisms highlights a significant gap that explainable ML tools could fill.

Automated Advisors

Wealthfront and Betterment use Modern Portfolio Theory (MPT) to generate diversified portfolios aligned with long-term investment goals. While effective for passive investing, these systems do not address short-term trading strategies nor provide action-level explainability.

Open-Source Algorithmic Tools

Freqtrade (2024) and QuantConnect provide powerful infrastructures for developing complex algorithmic strategies. However, these require programming knowledge and do not offer conversational interfaces or explanation engines, limiting accessibility.

Current Relevance to the Project

The Financial Advisor Bot aims to bridge the gap between:

- High complexity but low explainability systems
- Easy-to-use but non-transparent platforms

By combining RL, SMA signals, and natural-language explanations into a conversational interface, the project seeks to offer an interpretable decision-support system tailored to everyday investors. The shortcomings identified in existing tools have directly shaped the planned system requirements and design goals.

Summary and Identified Research Gap

The literature collectively highlights several gaps that the Financial Advisor Bot intends to address:

1. RL systems offer adaptive potential but lack transparency, making them unsuitable for non-expert users without additional explanation mechanisms.
2. Technical trading strategies such as SMA crossovers are interpretable, but their lack of adaptability limits performance in dynamic markets.
3. Backtesting tools are powerful but rarely accessible, and few are integrated into explainable advisory interfaces.
4. Current FinTech platforms prioritise usability over explainability, leaving users uncertain about how decisions are formed.
5. XAI approaches show promise, but no existing retail trading tool combines RL, classical technical analysis, and explanation-driven outputs in a unified system.

These gaps justify the development of a hybrid, explainable advisory system that balances interpretability with adaptive intelligence. The insights from the reviewed literature continue to guide the project's architecture, prototype development, and evaluation plans.

Design

Overview of the Project

The Financial Advisor Bot is being designed as an explainable AI-driven decision-support system that assists retail investors by generating BUY, SELL, or HOLD recommendations for stock market assets. Rather than functioning as a fully autonomous trading platform, the system aims to support informed decision-making by translating technical indicators and machine-learning outputs into natural-language explanations.

The overall system design focuses on combining classical analytical methods - specifically the simple moving average (SMA) crossover approach - with a reinforcement learning (RL) agent that will be integrated later in the project. The design emphasises interpretability, accessibility, and modularity, ensuring that each component can be developed independently while contributing to the system's cumulative intelligence.

While the SMA prototype has been partially implemented, the reinforcement learning module, backtesting engine, and explanation system are still under development. This design chapter outlines the project's architectural vision, user and domain justifications, key technical choices, and the planned integration between modules as the system moves toward completion.

Template Used for the Project

This project adopts the Decision-Making Under Uncertainty design template, a framework ideally suited for systems where data is incomplete, market conditions evolve unpredictably, and actions must be chosen sequentially. Stock trading represents a

classic case of decision-making under uncertainty: future price movements cannot be fully predicted, and each BUY, SELL, or HOLD action affects later opportunities and risks.

The template influences the project in several ways:

- **State-Based Decision Logic:** Both SMA signals and the planned RL agent interpret market conditions as states transitioning over time.
- **Sequential Actions:** The agent learns from repeated decisions across many time steps, reflecting real-world trading behaviour.
- **Uncertainty Modelling:** Reinforcement learning accommodates noisy data and uncertain outcomes by adjusting its policy through experience.
- **Explainability Requirements:** Users need visibility into how decisions are made, especially when dealing with unpredictable financial environments.

This template provides a conceptual foundation for guiding data flows, strategy interactions, and evaluation methods throughout the remaining development phases.

Domain and Intended Users

The domain of the project is retail investing, with a specific emphasis on U.S.-listed equities such as Apple (AAPL), Tesla (TSLA), and Netflix (NFLX). These stocks are frequently traded, highly liquid, and provide rich historical datasets suitable for training and validating algorithmic strategies.

Intended Users

1. **Retail Investors**

Individuals with limited financial expertise who seek clear, interpretable guidance rather than opaque algorithmic predictions.

2. **Students in Finance, Computing, or Data Science**

Users interested in learning how machine learning and technical analysis interact within real-world decision-making systems.

3. **Beginner Traders**

Users who want basic explanations of trading signals alongside performance insights.

4. **Researchers and Developers**

Individuals exploring human–AI collaboration or explainable financial models.

User Requirements

Preliminary user analysis suggests that the system must prioritise:

- **Interpretability:** Users require explanations they can understand without expertise.
- **Simplicity:** A conversational interface reduces cognitive load.
- **Accuracy:** Signals must be reliable enough to support learning and decision-making.
- **Transparency:** Users should see how decisions are produced.
- **Accessibility:** The web interface should work on mobile and desktop devices.

These requirements directly inform the system's design choices, such as the early implementation of SMA (simple and interpretable), the planned NLG-based explanation engine, and the chat-style frontend.

Justification of Design Choices

The system's design is driven by both technical considerations and user needs.

Hybrid Decision-Making Engine

A combination of SMA crossovers and reinforcement learning is planned because:

- SMA signals are simple, interpretable, and form a strong baseline.
- RL introduces adaptability and learning from experience.

- Hybrid models can stabilise performance by offsetting the weaknesses of individual approaches.

Explainability as a Core Requirement

Explainability is essential for retail users who may not trust or understand algorithmic predictions. The project therefore prioritises:

- Template-based NLG for reliability
- Optional LLM integration for conversational fluency
- Confidence scoring for transparency

Lightweight, Modular Architecture

A modular architecture allows each component to evolve independently, reducing development risk. This is particularly important in a multi-phase project where SMA is implemented early, RL follows later, and UX design continues in parallel.

Web-Based Interface

A web interface ensures easy access across devices without requiring installation. A chat-style design mirrors tools users are already familiar with, such as messaging apps or customer-service bots.

Python + React Tech Stack

- Python facilitates machine-learning workflows.
- Flask provides an easy way to expose ML functions via REST APIs.
- React enables a fluid and responsive user experience.
- Tailwind CSS speeds up interface styling.

These design justifications ensure that both the technical and user-centred goals of the project are aligned.

Overall System Architecture (Planned)

The Financial Advisor Bot's system architecture consists of six interconnected layers. At this stage, the architecture is being implemented incrementally, beginning with the data layer and SMA module.

Data Layer

This layer retrieves and preprocesses historical stock data.

- **Source:** Yahoo Finance API (via yfinance)
- **Data:** OHLCV (Open, High, Low, Close, Volume)
- **Processing:** 6-month intervals, converted into Pandas DataFrames
- **Status:** Implemented and functional

This layer provides inputs to both the strategy engine and planned backtesting module.

Strategy Engine

The engine is designed to produce signals using two complementary methods:

1. SMA Crossover Strategy (Implemented Prototype)
    - Uses 10-day and 50-day moving averages
    - BUY signal when SMA10 crosses above SMA50
    - SELL signal when SMA10 crosses below SMA50
    - HOLD when no crossover is present

This module forms the working feature prototype.

2. Reinforcement Learning Agent (Planned)
    - Tabular Q-learning

- State representation based on relative SMA positions and price trends
- Actions: BUY, SELL, HOLD
- Rewards: profit/loss from transitions
- Training: multiple historical episodes

This module is under early development and will be integrated in later stages.

Explanation Engine (Planned & Partially Designed)

A two-tier explanation engine is being designed:

1. Template-Based Generator
   - Provides structured, deterministic explanations
   - Maps signals + confidence scores to predefined templates
2. LLM-Enhanced Mode (Optional Future Integration)
   - Utilises Ollama or similar for richer language
   - Will be used only after ensuring accuracy and safety

The explanation engine will become the core of the system's interpretability.

Backtesting Module (In Progress)

The backtesting engine will simulate strategy performance across historical data:

- Executes trades based on signals
- Tracks portfolio value
- Computes performance metrics (win rate, drawdowns, return percentage)
- Produces equity curves for visualisation

A basic simulation loop has been drafted, but integration and metric computation are still in development.

Backend Layer (Planned Implementation)

The backend will be powered by Flask:

- Exposes /advice and /evaluate endpoints
- Handles communication between frontend and strategy engine
- Maintains modularity by separating ML logic from UI

Initial scaffolding exists, but most logic will be added when RL and backtesting are complete.

Frontend Layer (Early Mock-Up Implemented)

A React-based interface is being designed:

- Chat-style message flow
- Input box for stock symbols
- Display of generated signals and explanations
- Slide-up evaluation panel (planned for later)

The current version is a simple prototype showing the interaction pattern.

Identification of Key Technologies and Methods

The following technologies will be used across project components:

Machine Learning / AI

- Q-learning (Tabular): Chosen for simplicity and transparency
- Technical Indicators: SMA primarily, with potential expansion
- Reward-Based Learning: For decision optimisation

Backend & Data Processing

- Python 3.10
- Flask for REST APIs
- Pandas & NumPy for data manipulation

- yfinance for market data retrieval

Frontend & UX

- React.js
- Tailwind CSS
- Recharts (planned) for visualising equity curves

Evaluation Methods

- Backtesting methodology
- Signal consistency checks
- Interpretability analysis
- User experience testing (later stage)

Workplan (Gantt Chart Summary)

The project is structured into distinct phases:

Weeks 1–4: Research & Requirements

- Conduct literature review
- Clarify system requirements
- Select project template (Decision-Making Under Uncertainty)
- Draft initial architecture

Weeks 5–6: SMA Prototype Development

- Implement SMA10/50 crossover
- Test basic signal outputs
- Validate correctness visually and numerically

Weeks 7–8: Reinforcement Learning Agent

- Develop preliminary Q-learning agent
- Define state and reward structure
- Begin early experiments

Weeks 9–11: Backtesting Engine & Explanation System

- Build simulation loop
- Compute performance metrics
- Design template-based NLG explanations

Weeks 12–14: Frontend Integration

- Connect backend to React interface
- Implement chat interactions
- Add evaluation panel

Weeks 15+: Evaluation & Refinement

- Compare SMA vs RL
- Conduct interpretability testing
- Write final report

This staged plan ensures progress across components while allowing iterative refinement.

Planned Evaluation Strategy

The evaluation strategy is designed to determine both the technical and user-centred effectiveness of the system.

1. **Strategy Evaluation**
    - Compare SMA and RL performance
    - Use metrics such as returns, win rate, and drawdowns
    - Conduct walk-forward testing to reduce overfitting
2. **Explanation Evaluation**
    - Evaluate clarity and correctness of template-based explanations
    - Later evaluate fluency of LLM-enhanced explanations
    - Collect informal user feedback to assess interpretability

3. **Usability Evaluation**
    - Test the web interface with sample users
    - Assess ease of use, responsiveness, and clarity
    - Identify UX improvements
4. **Combined System Evaluation**

Once RL and NLG are integrated, full-system evaluations will analyse:

- Accuracy vs interpretability trade-off
- Trust and confidence reported by users
- Performance stability across different market conditions

Feature Prototype

Overview of the Implemented Feature

At this stage of the project, a feature prototype has been partially developed to demonstrate the feasibility of the system's core trading logic. The chosen feature - the Simple Moving Average (SMA) crossover strategy - serves as the foundational analytical component of the Financial Advisor Bot. This prototype enables the system to fetch historical stock data, compute short-term and long-term moving averages, and generate preliminary BUY, SELL, or HOLD recommendations based on crossovers between the two lines.

Although this strategy represents only one part of the planned hybrid architecture, it provides a practical and interpretable starting point for testing the data pipeline, validating signal generation principles, and assessing integration with backend components. The SMA crossover is intentionally used as the first feature because of its interpretability, its prevalence in retail trading tools, and its suitability as a baseline for evaluating future reinforcement learning signals.

This chapter outlines the prototype's implementation details, demonstrates its operation, discusses the preliminary evaluation of its feasibility, and identifies improvements planned for subsequent phases of development.

Rationale for Selecting SMA as Initial Prototype

The SMA crossover strategy was selected as the first implemented feature for several reasons:

1. Interpretability

Retail investors and novice traders often rely on moving averages to understand market momentum. The SMA crossover rule - BUY when the short-term average crosses above the long-term average - is intuitive and widely understood. This aligns with the project's commitment to explainable outputs.

2. Lightweight and Easy to Validate

Compared with early-stage reinforcement learning models, SMA calculations are computationally simple and allow fast testing. This is important during the prototyping phase, where the goal is to validate the data retrieval pipeline and signal logic before integrating more complex models.

3. Baseline for Future Comparison

The SMA-generated signals form the control benchmark against which the future RL agent will be evaluated. This helps determine whether reinforcement learning truly adds value beyond classical indicators.

4. Ideal for Partial System Testing

Since SMA signals do not require complex training loops or parameter tuning, they allow early testing of:

- API endpoints
- Data formatting
- Visualisation plans
- Integration with the future explanation engine

This prototype therefore represents a strategically chosen first step in the broader project roadmap.

Implementation Details

The SMA crossover prototype consists of three primary components:

1. Data Retrieval Module
2. SMA Calculation Module
3. Signal Generation Logic

Data Retrieval

The system uses the yfinance library to fetch six months of historical daily price data for a given stock symbol. This includes:

- Open price
- High price
- Low price
- Close price
- Trading volume

The retrieved dataset is cleaned and transformed into a Pandas DataFrame. Only the closing prices are used for SMA calculations due to their conventional role in momentum-based trading analysis.

The data retrieval pipeline has proven stable across several symbols (e.g., AAPL, TSLA, NFLX), and preliminary error handling has been implemented for invalid or unsupported tickers.

SMA Computation

The SMA calculations are implemented using Pandas' rolling-window functionality:

- **SMA10 (Short-Term):**
  Represents the average closing price over the past 10 trading days.

- **SMA50 (Long-Term):**

  Represents the average closing price over the past 50 trading days.

These two averages are added as new columns to the DataFrame. Edge cases - such as the first 50 rows where SMA50 is undefined - are handled by excluding early periods from signal analysis.

The resulting dataset provides a time series of short-term and long-term averages forming the basis for crossover signal detection.

Signal Generation Logic

The SMA crossover logic is implemented as follows:

- **BUY Signal:**

  Triggered when SMA10 crosses above SMA50

  → Suggests upward momentum and potential short-term price increase.

- **SELL Signal:**

  Triggered when SMA10 crosses below SMA50

  → Indicates potential downward reversal.

- **HOLD Signal:**

  Issued when neither of the above conditions is met.

To detect crossovers, the system compares the current and previous positions of SMA10 relative to SMA50. This prevents false signals that might arise if SMA10 and SMA50 temporarily touch or fluctuate without clear directional change.

The output is structured as a dictionary containing:

- Signal type (BUY/SELL/HOLD)
- Recent SMA values
- Timestamp
- Basic reasoning to be later expanded by the NLG module

At this stage, the explanation is still generic, but in later phases it will be transformed into natural language via the explanation engine.

Demonstration of the Prototype

The SMA feature prototype has been tested using well-known stock tickers. While the frontend is still in early development, the backend returns structured JSON outputs to confirm correct signal generation.

Example (AAPL):


```
{
  "signal": "BUY",
  "sma10": 168.24,
  "sma50": 165.17,
  "reason": "Short-term SMA crossed above long-term SMA."
}
```


Example (TSLA):


```
{
  "signal": "SELL",
  "sma10": 214.72,
  "sma50": 227.45,
  "reason": "Short-term SMA crossed below long-term SMA."
}
```

These outputs demonstrate that:

- The SMA logic operates correctly on multiple tickers
- BUY/SELL/HOLD states are generated according to classical trading rules
- The data retrieval and indicator computation pipeline functions reliably

While no visualisation is implemented in the frontend yet, backend tests confirm the feasibility of building evaluation charts once the backtester is fully developed.

Early Evaluation of the Prototype

The evaluation of the SMA crossover prototype is still in an early stage and focuses on feasibility, correctness, and interpretability rather than performance optimisation.

Functional Correctness

Manual inspection of generated signals across multiple trading periods indicates that:

- BUY signals generally occur during upward-trending regions
- SELL signals occur during downward shifts
- HOLD signals dominate during sideways markets, which is expected
- No contradictory signals were observed on the same timestep

This suggests the crossover logic is implemented correctly according to theoretical SMA behaviour.

Preliminary Backtesting Feasibility

A partial version of the backtesting engine has been used to simulate the SMA strategy on selected symbols. While the module is still under development, initial tests confirm that:

- Trades are executed at appropriate crossover points
- Portfolio value calculation is functioning
- Equity curves can be extracted for future frontend visualisation

However, more complex metrics (e.g., Sharpe Ratio, max drawdown) are not yet calculated.

Interpretability Assessment

From a user perspective, the SMA strategy provides inherently interpretable signals. Even without the final explanation engine, the reasoning behind each signal is clear:

- "Short-term trend rising above long-term trend" → BUY
- "Short-term trend falling below long-term trend" → SELL
- "No strong trend signals" → HOLD

This clarity reinforces the SMA strategy's suitability as an introductory prototype for retail users.

Limitations Identified During Testing

Several limitations emerged during early testing:

1. **Lagging Behaviour of SMA**
   Moving averages inherently lag behind price movements, causing late BUY/SELL signals in fast-moving markets.
2. **Choppiness in Sideways Markets**
   SMA crossovers can produce false signals during low-volatility periods.
3. **Lack of Confidence Scoring**
   Signals currently do not quantify strength or reliability.
4. **Partial Backtesting Capabilities**
   Without full evaluation metrics, performance assessment is incomplete.
5. **No Integration with the Explanation Engine Yet**
   Explanations remain basic and require transformation into user-friendly narrative text.

These limitations are expected at this stage and form the basis for planned improvements.

Planned Improvements

The SMA prototype is functioning as intended, but several enhancements are planned to strengthen its reliability, integrate it into the full system, and prepare it for comparison with the reinforcement learning agent.

1. Integration with Confidence Scoring

A confidence score will be added based on:

- Magnitude of the difference between SMA10 and SMA50
- Volatility of recent price movements
- Slope direction of both averages

This will later feed into the explanation generator.

2. Completion of Backtesting Engine

The backtesting module will be expanded to include:

- Win/loss ratio
- Percentage return
- Maximum drawdown
- Trade frequency
- Equity curve visualisation

These metrics will be used to evaluate the SMA strategy and later compare it with RL performance.

3. Integration with the Explanation Engine

The system will begin generating structured explanations such as:

- “The short-term moving average has risen above the long-term average, indicating emerging bullish momentum.”

This will later evolve into more natural-sounding text once the LLM-enhanced mode is implemented.

4. RL Agent Comparison

Once the reinforcement learning model is integrated, the SMA output will act as:

- A baseline for comparative testing
- A contextual feature for the RL state space
- A fallback in situations where the RL agent has low confidence

5. Move to Frontend Visualisation

The React interface will eventually display:

- Crossover points on charts
- Interactive trade logs
- Evaluation metrics

The SMA prototype's output will be formatted into user-friendly UI components.

Summary

The implementation of the SMA crossover strategy represents a successful first milestone in the Financial Advisor Bot project. It validates the core data-processing pipeline, confirms the ability to generate interpretable trading signals, and establishes the baseline logic against which more sophisticated models will later be evaluated.

Although still limited in functionality and scope, the SMA prototype provides the structural and conceptual foundation for the reinforcement learning agent, explanation engine, and full backtesting module planned in the next development stages.